

From Pilot Evidence to Conditional Scale: An AI4SE Operating Stack for Mid-to-Large Organizations

Juan Qiu*, Qilong Yan*

*Inspire Group, China

Emails: juan.qiu@inspiregroup.com, qlyan@inspiregroup.com

Abstract—Mid-to-large software organizations—multi-site engineering groups with enough process inertia and cross-team handoff that a tool trial does not diffuse by itself—often equate an AI-for-software-engineering (AI4SE) assistant rollout, or a completed pilot increment, with readiness to scale. As generation cost falls, scarce capacity shifts from coding throughput to *definition and verification*—knowing what is correct and proving it. We present an AI4SE operating stack for that setting—a layered reading of classical software engineering together with Lean/Agile team modes whose primary value object shifts from stories toward specification-driven *change requests*—and an Explore, Consolidation, and conditional Scale adoption rhythm in which stage closeout and admission are evidence reviews rather than calendar events. At Org-A, a six-week Explore engagement produced a citable method, countable opportunity assets, and a readiness-labeled executable-asset ledger; management met Explore closeout and opened Consolidation, but did not admit Scale. As a secondary recognition observation, Org-B—a manufacturing enterprise whose incumbent process is Integrated Product Development (IPD)—selected the same stack and rhythm after co-located alternatives were judged infeasible and is now in a mid-July to mid-September 2026 Explore that remains open. The paper contributes the operating stack and the Org-A stage-and-asset record; Org-B supplies only a decision-fact vignette, not a second completed case. It does not claim organization-wide scale, completed Scale admission, or platform-measured return.

Index Terms—AI for software engineering, AI4SE operating stack, industrial experience report

I. INTRODUCTION

Sponsors of AI4SE programs face a recurring management error: equating access to coding assistants, or completion of a facilitated pilot, with a scale-ready organizational capability. The first produces local tool use; the second may produce a one-off demonstration. Neither, by itself, answers whether a new team can run a complete cycle under agreed guardrails. Guidance on developer productivity already warns that a single activity or throughput slogan is a poor decision aid [1]. A sharper industrial reading is that AI coding assistance compresses *implementation* cost while amplifying errors that

originate in vague intent, weak specifications, and unverified agent output: the bottleneck moves toward *definition scarcity* and verification capacity. The corresponding transformation error is to treat a well-received pilot as authorization for organization-wide diffusion.

We present an AI4SE operating stack—and an Explore–Consolidation–Scale adoption rhythm—for advancing industrial AI4SE without treating calendar completion as admission—validated primarily by Org-A’s recorded Explore closeout and Scale-admission dispositions and asset ledger, not by a method brochure alone. A stage has a purpose, work products, and an observable exit that is also an admission test for the next stage. Explore asks whether a method can be exercised end-to-end in a real delivery context. Consolidation asks whether the pilot team can repeat and teach that method. Scale, entered only conditionally, asks whether a new team can complete a cycle independently. Org-A supplies the primary longitudinal evidence; Org-B supplies only a bounded recognition vignette (local adoption after infeasible alternatives, Explore still open). The transferable object is the stack-plus-stage-criteria contract—not a tool catalog and not a completed rollout.

We organize the experience around three questions:

- **Q1.** How can an industrial AI4SE path from pilot toward scale be designed (goals, deliverables, and closeout, admission, and exit tests)?
- **Q2.** What assets, signals, and admission boundaries did Org-A’s completed Explore produce in support of Consolidation and of only-conditional Scale?
- **Q3.** While Org-B is still in Explore, what does a local adoption decision after infeasible alternatives show—and what does it not show?

Methodological stance. This manuscript is an industrial experience report [2]. Facilitators co-designed the path with client teams, but the claims we defend are those Org-A management actually decided (Explore closeout met / Scale admission held) and the readiness labels in the handoff ledger—practitioner-validated decisions, not a consultant self-score. Org-B remains a secondary recognition vignette [3]. We do not claim multi-case theoretical saturation, a completed design-science evaluate cycle, researcher-owned action-research agency, or completed Scale at either organization.

J. Qiu is Principal AI4SE Consultant at Inspire Group (Ph.D., Software Engineering, Tongji University; 18+ years in software engineering, R&D productivity, and enterprise architecture). Q. Yan is AI4SE Engineering Advisor at Inspire Group (M.S., Computer Science, Sichuan University; 10+ years in enterprise architecture, DevOps, and AI-agent engineering). Inspire Group is a technology-services firm formed from the former Thoughtworks China business after its acquisition by Hillhouse Capital (announced 2025); the Inspire authors facilitated the Org-A and Org-B engagements. Industrial partners remain anonymized (Org-A, Org-B); no client co-authors.

Contributions. (1) an AI4SE operating stack (Pressman-layer upgrade; Lean/Agile→team paradigm→SDD with story→change-request value objects) plus an Explore–Consolidation–Scale adoption rhythm with closeout/admission criteria (citable / runnable / transferable readiness; stub $\neq$ ready); (2) Org-A evidence that Explore completed, Consolidation started, and Scale admission was not met—stated as a three-layer feasibility argument rather than as assumed diffusion; (3) a bounded Org-B decision fact only: the same stack and rhythm were selected after alternative approaches were judged infeasible, under an IPD constraint, with Explore still open (not a second completed stage-criteria packet).

Bounded Explore-phase effectiveness and maturity signals appear here only as stage-criteria inputs with stated limits. A companion measurement manuscript (separate contribution; dual-lens labeled reporting of capability and delivery signals at Org-A) owns how those signals are stratified and later promoted from estimate to platform measurement; we do not reproduce its tables or claim concurrent review of that companion manuscript here.¹

Section II supplies the conceptual overlay. Section III defines the stages and closeout/admission criteria. Section IV reports Org-A. Section V reports Org-B. Sections VI–VIII discuss takeaways, threats, related work, and the claim ceiling.

II. CONCEPTUAL BACKGROUND

The operating stack and adoption rhythm are the contribution of this paper. This section supplies a compact vocabulary for *why adoption is staged* and *what the stack carries*. It is not a second framework to be evaluated on its own, and it is not a restatement of a practitioner slide deck.

A. Domain Fit: Why Exploration Is Not Instant Scale

Snowden and Boone’s Cynefin framing distinguishes decision domains in which best practice, expert analysis, or probe-and-sense behavior is appropriate [4]. Early industrial AI4SE work is typically *complex*: workable method is emergent, so Explore is a probe that must produce inspectable method assets rather than a purchased best-practice pack. Consolidation moves durable findings toward the *complicated* domain: playbooks, harness hardening, and seed coaches support sense-analyze–respond transfer. Scale entry is admitted only when enough of the method is stable to diffuse without re-running full discovery—closer to clear or well-analyzed complicated work, never by calendar alone. The gates are therefore domain-fit devices: treating a Complex probe as Clear rollout is the failure mode this paper records as “pilot success without Scale admission.”

¹Companion status at freeze: authoring for a peer-reviewed software-engineering venue; unit of analysis = measurement scheme, not the operating stack and adoption rhythm. No Org-A workshop or maturity numbers from that manuscript are imported here.

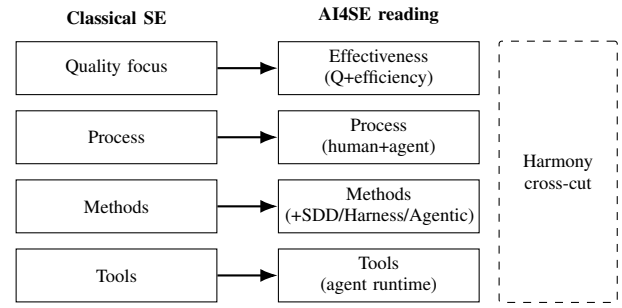


Fig. 1. Inherited layered software-engineering structure with upgraded AI4SE semantics (conceptual). Harness/SDD/Agentic sit primarily in Methods; Harmony cross-cuts rather than stacking a fifth layer.

B. Industry Fault Lines as Staging Rationale

Senior-practitioner syntheses from Thoughtworks’ Utah and Europe Future of Software Engineering retreats highlight fault lines that motivate staging [5], [6]: engineering rigor migrating into specifications, tests, and constraints; a supervisory *middle loop* between coding and delivery; verification (not generation) as scarce capacity; harness ownership around the agent; and a two-clock mismatch in which code-production time collapses while decision time does not. We use that map only as industrial *rationale*. Explore must freeze probe evidence; Consolidation must harden verification, harness ownership, and seed coaches; Scale is admitted only when decision and verification capacity can absorb a receiving team. Retreat anecdotes are not Org-A/Org-B facts, and retreat consensus is not empirical validation of our stage criteria; denser positioning appears in Section VIII.

C. Layer Inheritance: Pressman Structure, Upgraded Semantics

Classical software-engineering textbooks present a layered view in which a quality focus grounds process, methods, and tools, and in which more visible layers depend on less visible ones [7]. Figure 1 states our industrial reading: the *structure* is inherited; the *semantics* must be upgraded for human–agent delivery. Quality focus becomes *Effectiveness* (quality plus waste-aware efficiency). Process, methods, and tools retain their roles, but executors are humans *and* agents. Harness Engineering, Spec-Driven Development (SDD), and Agentic Engineering are placed in the *methods* family (discipline and assets), not as a replacement for the process layer or as a tools-only purchase. *Harmony* is a cross-cutting human–agent contract, not a fifth stacked layer.

D. Operating Stack: Value Layer, Team Paradigm, and SDD

Pressman’s layers answer *what* to upgrade; delivery teams still need an *operating stack* that says how Lean/Agile work continues under AI4SE. Figure 2 states three nested layers we used with sponsors and coaches. The top layer keeps Lean/Agile value principles (flow, pull, feedback, respect for people). The middle layer keeps classical team operating modes—Scrum, Kanban, or Scrumban—as the cadence and role container (Org-A primarily Scrum; Org-B combines IPD

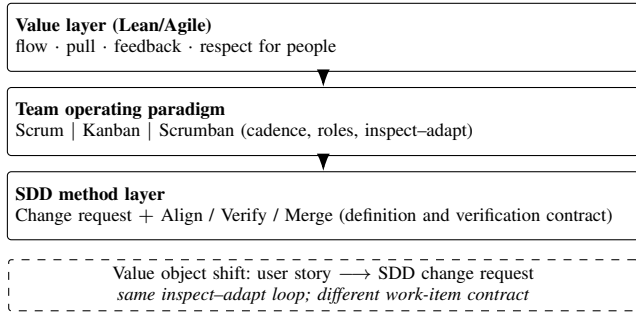


Fig. 2. Three-layer AI4SE operating stack. Classical Lean/Agile team modes remain; the primary value-flow object moves from stories toward SDD change requests.

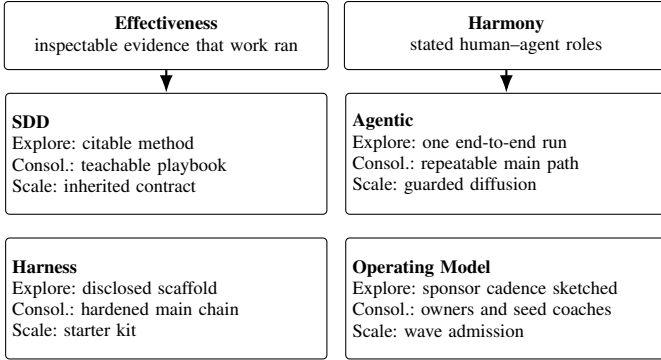


Fig. 3. Effectiveness, Harmony, and four capability directions mapped onto the operating stack and adoption rhythm. The overlay is conceptual; it is not a second evaluated contribution.

with multiple team modes). The bottom layer is Spec-Driven Development (SDD): Align, Verify, and Merge reviews that protect each change under the chosen paradigm p .

The industrial claim is deliberately narrow: those team modes remain effective; what shifts is the primary *unit of value flow*. Where boards and backlogs once centered on user stories, SDD centers on a citable *change request* (and its Align→Verify→Merge contract) as the object that carries definition and verification evidence. Stories and other backlog items may still exist; they no longer substitute for an inspectable change contract when agents draft and apply increments. We reserve the word *gate* for those process-level Align/Verify/Merge reviews in the method pack, and we use *closeout / admission* for organizational stage decisions in Section III—so the two loops do not share G1/G2 numbering.

E. Effectiveness, Harmony, and Four In-Path Directions

We use two cross-cutting concerns—*Effectiveness* and *Harmony*—and four capability directions: SDD, Agentic Engineering, Harness Engineering, and an AI4SE Operating Model. Figure 3 maps them onto the stages of Section III. We do not treat these labels as a maturity model, a vendor stack, or a substitute for stage closeout and admission.

Effectiveness is the requirement that each stage produce inspectable evidence that the selected work actually ran under local constraints—a citable method, a validation-vehicle run-through, or a new-team cycle—rather than attendance counts

or tool-license counts. Harmony is the requirement that human and agent accountability be *operationalized*, not merely named. In the engagements we distinguish four complementary functions that may be held by fewer people in a small team but must not be vacant: *intent ownership* (what to build and what success means), *specification stewardship* (living specs and change deltas as the shared source of truth), *AI-asset orchestration* (skills, recipes, and harness units that make the method executable), and *verification leadership* (acceptance and human sign-off). A working rule is *contract separation*: the party that proposes or applies a change must not be the sole party that verifies it—agents may draft and execute, but humans retain intent and verification authority. Both Effectiveness and Harmony are inputs to Explore closeout and Scale admission. They are not outcome multipliers, and this paper does not report them as measured organizational return.

The four directions are stage-relative obligations.

SDD converts intent into specifications that agents and humans can execute, review, and accept. Explore must produce a first citable method; Consolidation must make that method teachable; Scale uses it as the contract a receiving team inherits. Spec-driven development paired with agentic execution is an active industrial theme; Gupta discusses the pairing in a preprint on high-accuracy agentic frameworks [8]. We cite that work as thematic context, not as an evaluation of Org-A or Org-B.

Agentic Engineering turns those specifications into composable, reviewable agent work—skills, commands, and recipes—rather than ad-hoc prompting. Explore needs one end-to-end agentic run; Consolidation needs the principal path to be repeatable by the pilot team; Scale admits that path only with declared guardrails.

Harness Engineering is the runtime—context, tools, permissions, feedback, and audit—in which agents operate. Explore may leave the harness as a disclosed scaffold; Consolidation must harden the main chain; Scale requires a starter kit another team can run. A stub is not a production harness; Section IV operationalizes that distinction in the Org-A ledger.

Operating Model covers owners, coaching, sponsor cadence, and wave admission—including named holders of the Harmony functions above. It is sketched in Explore, staffed in Consolidation (named owners and seed coaches), and enforced at Scale entry. Without it, a hardened harness still has no authorized receiving team.

F. Certainty Boundary: User Harness, Model, and Method Assets

Industrial harness writing emphasizes bounded contexts among users, coding agents, and models [9]. Figure 4 is our AI4SE reading of that idea for transformation work: organizational certainty does not live in the model alone. We write the staged enablement object as a paradigm-conditioned composition

$$\mathcal{H}^P(x) = \text{UserHarness}^P(\text{CodingAgent}^P(\text{Model}(x))), \quad (1)$$

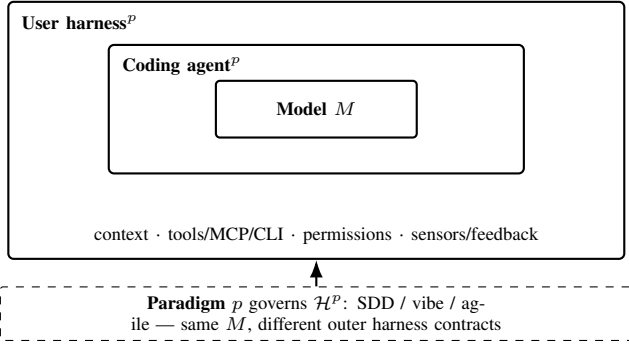


Fig. 4. Certainty boundary for AI4SE enablement (conceptual adaptation of harness bounded-context thinking [9]). Top: nested layers realizing (1). Bottom: paradigm p selects the outer contracts (Eq. (2)); method assets feed those outer units. Agentic engineering composes them—it is not a competing fourth paradigm.

where p is the governing delivery paradigm (for example spec-driven development versus loosely guided “vibe coding,” or an agile variant). The inner Model is comparatively stable across engagements; the outer two layers are where Explore must freeze inspectable structure and Consolidation must harden ownership. The same model under different p therefore yields different $\mathcal{H}^p$: an SDD-oriented user harness privileges contracts, acceptance checks, and reviewable agent recipes, whereas a vibe-oriented harness privileges fast prompt–edit loops and lighter formal Align/Verify/Merge checks. Neither is universally “correct”; the stage packet must record which p was chosen and what evidence that choice requires.

Operationally, each outer layer is itself a composition of smaller units around a frozen model operator M :

$$\mathcal{H}_{\text{single}}(x) = h_k(\cdots h_1(M(h_0(x))) \cdots), \quad (2)$$

with h_0 assembling context and guides, h_1 parsing and schema checks, and later h_i covering tools, permissions, sensors, and feedback. *Agentic engineering* is not a fourth paradigm competing with SDD or vibe coding; it is the engineering style that turns a chosen p into composable, executable, and verifiable AI assets—skills, subagents, commands, rules, and hooks—inside UserHarness^p and CodingAgent^p . Method assets (playbooks, SDD contracts, starter kits) feed those outer layers; they do not replace the model. The harness must remain an open class across SDD stacks (for example OpenSpec-, Superpowers-, or Spec-Kit-style frameworks, and future peers), not a lock-in to one template—a constraint Org-B makes explicit in Section V.

Skipping Consolidation typically leaves SDD prose and a stubbed harness in the hands of a new team—the failure mode Section III names as equating a pilot with scale. The conceptual layer therefore explains *why* stage criteria ask for method, asset, ownership, and transfer evidence; it does not add a parallel transformation method.

III. ADOPTION RHYTHM: EXPLORE TO CONDITIONAL SCALE

A. Why Closeout and Admission, Not a Calendar, Advance the Path

We define an AI4SE transformation path as a sequence of capability-building stages separated by explicit management decisions. A stage has a purpose, work products, and an observable exit condition. Its exit condition is also an admission test for the next stage: elapsed time can trigger a review, but cannot substitute for evidence. This distinction is important because a pilot can finish on schedule while leaving its method tacit, its automation fragile, or its ownership unresolved. Collapsing code-production time does not by itself collapse decision and verification time—another reason reviews must examine evidence rather than calendars.

The transformation path nests two control loops. *Stage closeout and admission decisions* decide whether the organization may enter Consolidation or admit a Scale wave. Inside delivery work, shorter *Align/Verify/Merge* reviews—align a delta specification, verify a candidate increment, then merge and archive—protect each change under the chosen paradigm p . Stage criteria ask whether method, assets, and ownership are transferable; Align/Verify/Merge reviews ask whether a particular change is definition-complete and evidence-backed. Confusing the two is a common failure: a team that passes local change reviews can still fail Scale admission if receiving teams cannot inherit the method.

The path instantiated at Org-A comprises *Explore*, *Consolidation*, and *Scale* (Fig. 5). Explore asks whether the method can be exercised end-to-end in a real delivery context. Consolidation asks whether the resulting method can be repeated and taught by the pilot team. Scale, which is entered only conditionally, asks whether a new team can run a complete cycle independently while retaining agreed guardrails. Outputs therefore flow forward as dependencies: the first-generation playbook and pilot assets feed hardening; the hardened method, executable starter kit, role owners, and seed coaches then support controlled diffusion.

B. Stages, Deliverables, and Exit Criteria

Stage 1—Explore. The purpose of Explore is feasibility, not broad deployment. In the Org-A instantiation, the indicative duration was approximately six weeks, spanning diagnosis and scenario selection, four workshop weeks over twelve opportunity themes, co-creation of an initial playbook and executable-asset scaffold, and use of a pilot change request as a validation vehicle. The exit criterion was an end-to-end run-through of the method chain. It was deliberately not completion of every feature in the validation vehicle.

Stage 2—Consolidation. Consolidation is the bridge between one successful run and a transferable organizational capability. Its indicative Org-A window was one to two months. The work comprises revising clauses that read correctly but do not run in practice, hardening the principal executable path from stubs to usable assets, assigning role ownership,

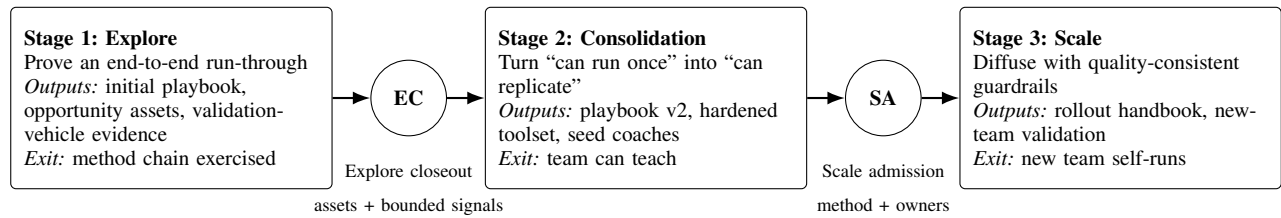


Fig. 5. Explore-Consolidation-Scale adoption rhythm with Explore closeout (EC) and Scale admission (SA) criteria. The figure specifies evidence required to advance; it does not indicate that Org-A completed Scale.

and rehearsing coach-the-coach transfer. Its intended products are a standardized playbook v2, a hardened toolset, and seed coaches. Hardening follows two practical rules drawn from the enablement practice: *do not promote immature automation* (skills and recipes must stabilize before they are wrapped as autonomous agents), and *keep apply separate from verify* (an execution agent must not also issue the verification verdict). The exit condition is “team can teach”: pilot-team members can explain and transfer the method, rather than only reproduce a consultant-led demonstration.

Stage 3—Scale. Scale is quality-consistent, wave-based diffusion rather than simultaneous organization-wide rollout. Admission requires the Consolidation exit, named role owners, a starter kit for sibling teams, and reserved sponsor cadence and resources. Seed coaches then enable successive teams and review practice consistency. The terminal evidence is a fresh team’s independent completion of a cycle under the guardrails. Thus, a calendar target can prompt a Scale-admission review, but unmet criteria block admission.

C. Closeout and Admission Semantics

Explore closeout is a *method-asset closeout*. Management asks whether a playbook can be shown and cited with its gaps visible; whether opportunity-point specifications and best practices can be inventoried; whether executable assets are distinguished as ready or stub; whether effectiveness and maturity signals carry their measurement boundaries; and whether the Consolidation-to-Scale path is concrete enough to discuss. A positive Explore closeout therefore licenses method hardening, not broad rollout. Table I makes this decision contract explicit.

Scale admission is stricter because it concerns transfer beyond the pilot setting. It tests whether know-how has moved from individuals into teachable guidance, executable assets, role ownership, and an enablement mechanism—and whether verification authority remains human-held under the nested Align/Verify/Merge reviews. The resulting admission decision is reversible and wave-specific: if a new team cannot operate with the starter kit and bounded coaching, the organization returns the deficient asset or role arrangement to Consolidation. This interpretation makes a stage decision a learning control, rather than a ceremonial approval checkpoint.

D. Operating Stage Criteria as an Evidence Review

A usable stage review needs a review packet, a decision owner, and a recorded disposition. The review packet should present the stage goal first and then map each promised output

to an inspectable artifact or observation. A readiness label accompanies each item so that “exists,” “runs on the primary path,” and “transferable to a new team” cannot collapse into one category. Known gaps remain in the packet as decision inputs. Removing them from view would make an approval easier to obtain but less useful for planning the next stage.

The decision owner evaluates sufficiency for the *next* stage, not perfection in the current one. At Explore closeout, an incomplete executable scaffold can be acceptable when it is disclosed and when the Consolidation plan names the hardening work. At Scale admission, the same scaffold is a blocker because a receiving team would depend on it. This relative interpretation prevents two common errors: rejecting useful Explore evidence because every asset is not yet finished, and admitting Scale because unfinished assets were tolerated at Explore closeout.

A stage disposition has three possible forms. *Advance* means the exit condition is met and the next stage may start under its declared scope. *Advance with conditions* records a narrow action whose completion is checked before a dependent activity begins; it must not be used to waive a core exit condition. *Hold* returns named gaps to the current stage. In all three forms, the evidence and rationale remain available for the next review. This creates continuity across decisions and makes negative evidence—for example, a stubbed primary path or missing owner—actionable rather than embarrassing.

The review also separates two kinds of uncertainty. Product uncertainty asks whether AI4SE can improve the selected work under local constraints; Explore reduces it through actual method use. Transfer uncertainty asks whether another team can reproduce that use without recreating the method; only Consolidation and a new-team test can reduce it. Treating these uncertainties separately explains why a positive Explore closeout and a negative Scale admission can be the expected, internally consistent outcome of the same review.

E. Three-Layer Scale-Feasibility Argument

The path permits a useful feasibility judgment before claiming completed diffusion. We structure that judgment in three layers.

Layer 1—the path is designed. Goals, work products, exits, and admissions are stated before scale readiness is assessed. This makes the claim falsifiable: missing ownership, hardening, or transfer evidence can stop advancement even when a pilot was well received.

TABLE I
DECISION TESTS AT THE TWO STAGE BOUNDARIES

Boundary	Evidence reviewed	Positive decision	What the decision does not imply
Explore closeout	Citable playbook with gaps stated; inventory of converged and observed opportunity assets; readiness of executable assets stated as ready or stub; bounded effectiveness/maturity signals; discussable next-stage path	The method chain ran end-to-end and the organization may enter Consolidation	All validation-vehicle features are complete; executable assets are ready for broad use; measured organization-wide return has been established
Scale admission	Team-can-teach evidence; hardened main execution path; named role owners; starter kit; sponsor rhythm and resources	A controlled wave may admit a new team with seed-coach support	The Explore pilot alone authorizes diffusion; every team is already independent; rollout outcomes are known in advance

Layer 2—Explore is validated end-to-end. For Org-A, the initial playbook, opportunity assets, and validation vehicle exercised the method chain under real delivery constraints. The evidence supports “can run once” and justified meeting Explore closeout; it does not establish repeatability across teams.

Layer 3—Scale remains conditional. Org-A’s pilot succeeded at Explore, but Scale admission was not met at closeout. Consolidation—including asset hardening and team self-solve capability—is the prerequisite for a later Scale decision. The feasibility judgment therefore rests on an explicit path, observed end-to-end use, a readiness-labeled asset ledger, and testable admission conditions. It does not rest on an assumed organization-wide return or on diffusion that has not occurred.

F. Anti-Patterns Exposed by the Criteria

Tool-only transformation treats access to a coding assistant or plugin catalog as the change itself. It omits the specifications, verification, context, roles, and learning routines needed to make tool use repeatable. Explore closeout exposes this weakness by requiring a citable method and bounded evidence; Scale admission exposes it by requiring transfer and ownership.

Pilot equals scale converts a successful demonstration directly into rollout authorization. It conflates end-to-end feasibility with cross-team replicability. Separating Explore closeout from Scale admission preserves the useful result of the pilot while making the missing admission evidence visible.

Skipping Consolidation carries first-generation prose and stubs directly into new teams. The likely burden then moves from explicit hardening to repeated local improvisation. The Stage-2 exit—team can teach—requires the organization to remove that burden before opening a Scale wave.

Agent self-certification lets the same generative path that applied a change also declare the change verified. Nested Align/Verify/Merge reviews and the Harmony verification function exist to forbid that shortcut: drafts and execution may be agentic; sign-off remains human.

IV. CASE ORG-A

A. Industrial Context and Evidence Basis

Org-A is an anonymized large global container-shipping and logistics enterprise with a substantial, multi-site in-house IT organization supporting commercial and operational systems.

The engagement was an approximately six-week AI4SE enablement pilot, not a scale-out program. The delivery setting used a multi-team agile model, shared requirements management, and distinct development, business-analysis, and quality roles. Quality remained a standing constraint: efficiency exploration was not allowed to relax the client’s existing quality controls.

We report the case as an industrial experience and preserve a chain from each case statement to a frozen evidence slice, following the traceability principle in case-study reporting [2]. The primary case record comprises the phase timeline, the stage-criteria definition, the L1–L4 handoff ledger, and post-closeout consolidation signals. We treat the playbook and executable assets as designed artifacts evaluated through use in their organizational environment, consistent with design-science evaluation logic [3]. This paper uses only qualitative signal boundaries from the evidence pack; measurement results are reserved for the companion measurement study.

B. Explore Activities

The Explore phase followed a six-block arc (Fig. 6). Kickoff in late June or early July 2026 established the diagnosis, scenario lock, baseline maturity exercise, and opportunity-theme plan. Workshop W1 (early July) addressed engineering foundations: harness/context, knowledge, and brownfield specification-driven development. W2 (mid-July) covered metrics and requirements entry and included a midterm management showcase. W3 (mid-to-late July) addressed review, agent validation, and test engineering. W4 (late July) ran two tracks in parallel: playbook v0.5 authoring and specification-based validation in the delivery track. An early-August executive readout closed Explore, aligned the Consolidation path, and explicitly did not claim Scale admission.

Across those four workshop weeks, the team worked through twelve opportunity themes and co-created initial method and opportunity assets. A pilot change request supplied the validation vehicle for using the method in delivery. This choice kept acceptance focused on whether the chain from opportunity through specification and practice could be exercised, rather than on shipping all business scope. The activity record consequently supports a process-feasibility conclusion, not a claim about broad adoption.

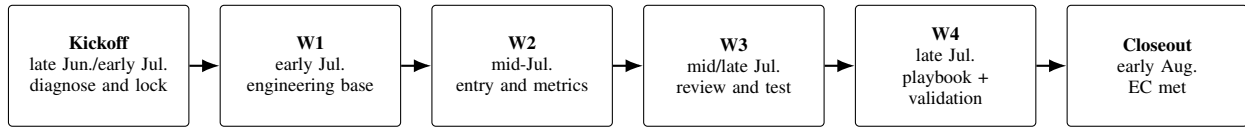


Fig. 6. Org-A Explore timeline (week-level windows). The early-August review met Explore closeout and opened Consolidation; it did not meet Scale admission.

C. From Activities to Stage Evidence

The workshop sequence was not treated as attendance evidence. Each block was expected to leave a trace in one or more asset layers. Foundation work informed the method guidance and executable scaffold; opportunity work created specifications and best practices; delivery-track use tested whether those descriptions could guide action; and the closeout evidence pack recorded both observed signals and their limits. This trace converted a series of facilitated sessions into inspectable inputs to Explore closeout.

The parallel tracks in W4 were particularly important to that conversion. Playbook authoring alone could have produced guidance that was internally coherent but detached from delivery. Delivery work alone could have produced a one-off result whose method remained tacit. Running authoring and specification validation together created a feedback loop: delivery exposed where wording or executable support was weak, while the playbook gave the team a stable object to revise and later cite. The closeout review then assessed the resulting artifacts and disclosed gaps rather than inferring capability from workshop completion.

We used three readiness distinctions when interpreting the handoff. *Citable* means an artifact is stable enough to identify and discuss. *Runnable* means that its principal path works in the intended local workflow. *Transferable* adds the ability of another team to use that path with declared guardrails and bounded coaching. Explore required a citable method and evidence of local use; Scale admission would require runnable and transferable support. The distinctions explain why the L1 playbook could satisfy its Explore-closeout criterion while the largely stubbed L3 pack could not satisfy Scale admission.

D. L1–L4 Outcomes and Readiness

The closeout inventory was organized into four layers (Table II). The layers distinguish descriptive guidance, reusable opportunity knowledge, executable support, and evidence. More importantly, the ledger records readiness separately from existence; a catalog entry is not treated as a deployable capability.

L1: citable method. The frozen v1.0 handbook captured the end-to-end method—including how intent becomes living specifications, change deltas, and accepted increments—well enough to be shown and referenced at closeout. Its frozen state made the Explore result auditable; it did not mean that Consolidation revisions were complete. Playbook v2 remained a Stage-2 target. A public method card distilled for inspection (chapter map, five first principles, Living-Spec artifact chain, 5+1 stages with Align/Verify/Merge Align/Verify/Merge reviews, paraphrased normative clauses, and declared v1 gaps)

is released with the data pack;² the full client-facing handbook text is not.

As an illustrative L1 extract (paraphrased, not a verbatim client clause): v1.0 required that every accepted increment cite a living-specification delta id, name a human verifier distinct from the apply agent, and refuse “generated-output equals verified” shortcuts. Org-A’s later Consolidation agenda explicitly included rewriting clauses that “read right but did not run” on the main L3 chain—a recurring failure mode for early playbooks. Non-ROI intensity at closeout was closeout-visible without claiming return: four workshop weeks plus kickoff/readout, twelve themes (ten released), 11+11 L2 assets, a largely stubbed L3 main chain, and Explore closeout met / Scale admission held.

L2: countable opportunity knowledge. The release set indexed 11 best practices and 11 corresponding specifications across twelve named themes: AI-ready story/living specification; UI prototyping; AI artifact review; cycle-time/AI-effectiveness measurement design; role handoff (observation); user harness/context pack; knowledge-base grounding; test engineering; token/model routing (observation); brownfield specification-driven development; agent nondeterminism evaluation; and CI toward AI control-system checks. Themes #5 and #9 retained observation status rather than being presented as fully converged, so management could count the handoff without erasing uncertainty within it.

L3: scaffold, not production readiness. The handoff contained a formal executable-asset pack whose redacted layout was: `commands/ (/sdd catalog), skills/{intent, spec, design, verify, harness}/, recipes/ (ordered stage sequences), adapters/ (coding-agent hosts), and agents/ (sample wrappers only)`. The catalog existed, but the main execution chain was largely stubbed: many skill bodies were scaffolds, and stage recipes were not yet a hardened primary path. Org-A therefore recorded the assets as present but did not infer production readiness from the directory tree, and did not treat early agent wrappers as mature automation. Moving the primary chain from stub to ready was explicitly assigned to Consolidation.

The validation vehicle was a pilot change request on a live delivery track: the team exercised intent capture, living-specification update, increment application, and human-owned verification under retained quality controls. Acceptance for Explore stayed on whether that method chain could be run and cited, not on shipping the vehicle’s full business scope. We do not report cycle-time or defect metrics for the vehicle here; those belong to the companion measurement manuscript.

²See `11-method-extract.md` under the Data Availability URL.

TABLE II
ORG-A EXPLORE ASSET LEDGER AT CLOSEOUT

Layer	Asset class	Observed outcome	Readiness boundary / Consolidation need
L1	Method playbook / operations handbook	Version 1.0 was frozen, citable, and sufficient for the Explore closeout (public method card released; full handbook withheld).	It was not playbook v2; clauses that read well but do not run remained targets for revision.
L2	Opportunity specifications and best practices	The release index contained 11 best practices and 11 specifications; opportunity themes #5 and #9 remained observation items.	Countability did not mean that every theme had converged.
L3	Executable plugins, skills, and recipes	The formal pack, including the /sdd path, skills, recipes, and adapters, was catalogued but largely stubbed on the main chain.	A stub is not production-ready. Main-chain hardening was Stage-2 work.
L4	Maturity/effectiveness evidence pack	The handoff included evidence pointers and signal-type boundaries.	The signals were process observations and expert estimates, not platform-measured return; this paper does not reuse their numeric results.

L4: bounded evidence. The evidence layer contained maturity and effectiveness materials and stated their signal types. Workshop judgments were expert estimates and the materials were not presented as platform-measured return. For this path paper, L4 establishes that evidence boundaries were available to the closeout review; it is not used to introduce the companion paper’s quantitative findings.

E. Phase-One Explore Closeout Decision

At the early-August executive readout, management applied five questions: whether the playbook was citable; whether opportunity assets were countable; whether executable readiness was labeled ready or stub; whether effectiveness and maturity signals carried boundaries; and whether the next-stage path could be discussed concretely. The result was a pass for the Explore closeout. The method had run end-to-end, the initial playbook and countable opportunity assets existed, and bounded signals could be reported.

The same review did *not* pass Scale admission. Replicability and seed coaches had not yet been demonstrated, the principal L3 execution chain remained largely stubbed, and the role, starter-kit, sponsor-cadence, and resource prerequisites were not all in place. This was not a contradictory verdict. Explore closeout answered whether the organization had enough evidence to harden a feasible method; Scale admission would answer whether that hardened method could be transferred to another team.

The two-part decision is the practical value of the path. Without distinct gates, management would have had to label the pilot either a failure because it was not ready for broad diffusion, or a scale success because it completed its planned activities. The stage-path account preserves the observed Explore achievement and the unresolved admission work at the same time.

F. Consolidation Started; Scale Entry Remained Conditional

After the early-August closeout, Org-A entered Consolidation, with an indicative one-to-two-month window from early August through the end of September 2026. The active work was framed as upgrading playbook v1 toward a standardized

v2 and hardening the L3 main chain. Seed coaches and “everyone can teach” were capability targets and the Stage-2 exit criterion; they were not reported as achieved at Explore closeout.

A consulting-side training and certification design also described Foundation, Practitioner, and Seed Coach tracks and a team-readiness certification concept. It was a cross-customer capability-preparation artifact at design-specification status: courseware, item bank, and learning platform implementation were pending. It therefore corroborates that scale enablement was being designed, but it is not evidence that Org-A executed training waves.

The resulting case posture is bounded but operationally useful. Layer 1 of the feasibility argument was in place because stages and closeout/admission criteria had been defined. Layer 2 was observed because Org-A completed an end-to-end Explore cycle and met Explore closeout. Layer 3 remained conditional because Scale admission had not been met: later Scale admission depended on the team-can-teach result, named owners, a hardened starter kit, and sponsor support. An indicative October calendar horizon was subordinate to those conditions. No new-team self-run, wave rollout, or organization-wide adoption was evidenced at the freeze point.

Org-A thus illustrates why movement from pilot evidence toward conditional Scale is a managed transition rather than a retrospective relabeling of pilot success. The case supplies evidence for feasibility, asset creation, and the start of hardening. It also supplies explicit negative evidence—stubs, missing transfer proof, and unmet prerequisites—that prevented an unsupported Scale claim. A later assessment can apply Scale admission to those same declared conditions without changing the meaning of the Explore result.

V. CASE ORG-B

A. Industrial Context and Evidence Basis

Org-B is an anonymized manufacturing enterprise. Unlike Org-A’s multi-team agile setting, Org-B’s incumbent product-development regime follows Integrated Product Development (IPD). The AI4SE path must therefore be adapted to IPD decision and lifecycle structure. We do not report a clause-level

mapping of Explore closeout or Scale admission onto Org-B’s IPD decision points; that mapping is a design constraint for Explore, not an observed process rewrite. We do not claim that Org-B has completed an organization-wide IPD×AI4SE rewrite.

The evidence basis is an author briefing aligned to the statement of work (SOW). There is no consultant-side workspace mirror: client work remains on the client intranet. Facts in this section are therefore SOW- and briefing-level, and we mark delivery status explicitly.

B. Adoption After Infeasible Alternatives

From about mid-July through mid-August 2026, other co-located consulting approaches led early exploration. Org-B concluded that those AI4SE transformation approaches were not feasible for its setting and selected the operating stack and adoption rhythm described here as the primary Explore method. We report this as a client decision outcome. We do not name the alternative providers, and we do not present a competitive scorecard.

From that transition through late August 2026, this team has been guiding multiple pilot teams through product research-and-development process analysis under the path. The calendar Explore window is mid-July to mid-September 2026. As of the manuscript freeze (late August 2026), Explore remains in progress. We do not write Org-B as Explore-closed, Consolidation-complete, or organization-wide scaled.

C. Multi-Mode Pilots under an IPD Constraint

The Explore wave intentionally samples heterogeneous software modes rather than a single representative team embedded software; internal-use application development; large, complex enterprise business systems; and AI-agent-oriented development. Mode diversity is a sampling design for this Explore wave, intended to exercise heterogeneous delivery constraints inside one organization. It is not a completed mode-by-mode outcome set, and it is not evidence of enterprise rollout or of a multi-enterprise sample.

D. Harness Intent, Scope, and Vignette Bound

The intended user-harness layer is SDD-oriented and non-exclusive: it should interoperate with Spec-Driven Development stacks of the OpenSpec / Superpowers / Spec Kit *class* (illustrative, not an allow-list) and remain extensible to peers, with MCP/CLI hooks into Org-B’s internal toolchain. We claim neither production-complete integration nor single-stack lock-in.

The SOW mixes Explore work with later-stage preparation: Goals 1–2 (method / harness toolkit and end-to-end playbook) are Explore targets still open at freeze; Goals 3–4 (training/certification and seed coaches) are scale-preparation items, not observed Consolidation/Scale outcomes; Goal 5 is measurement-scheme design only (no Org-B numbers here).

As a recognition vignette, Org-B shows only that an IPD manufacturer can record a local feasibility decision and start a multi-mode Explore under harness-portability constraints. It

TABLE III
ORG-A AND ORG-B ON THE SAME STACK AND RHYTHM

	Org-A	Org-B
Calendar	~6-week finished; started	Explore Consolidation
Process	Multi-team agile; retained quality controls	~2-month Explore, mid-July–mid-September; in progress
Pilot shape	Enablement plus validation vehicle	IPD incumbent; path must adapt
Adoption	Sponsored enablement; Explore closeout met / Scale admission held	Multi-mode team sample in one wave
Harness	Playbook and plugin pack in handoff; main chain stubbed	Method switch after infeasible alternatives
Evidence depth	In-repo ledger and stage packet	Extensible multi-SDD harness plus MCP/CLI; not locked to one stack
		Author briefing; no intranet mirror

does not show that alternatives failed in general, that Explore closed, that deliverables were accepted, or that Consolidation or organization-wide scale has started. Concrete negative process evidence at freeze: closeout/admission host reviews inside IPD are not yet mapped; that absence is an Explore design debt, not a completed IPD×AI4SE rewrite.

VI. DISCUSSION

A. Reading the Two Cases Together

Org-A and Org-B occupy different positions on the same path (Table III), with unequal evidence depth. Org-A is the primary case: Explore completed, Consolidation started, Scale admission withheld, with an in-repo ledger and stage packet. Org-B is a secondary recognition vignette: the path was selected after alternatives failed a local feasibility test, and Explore remains open under IPD. The shared path vocabulary can be named in more than one process heritage; that does not make Org-B a second completed case.

The contrast also clarifies what “method recognition” is allowed to mean. Org-B’s switch is a decision fact about local feasibility, not a proof that the path dominates unnamed alternatives in general, and not a claim that Org-B has already produced Org-A’s L1–L4 ledger. IPD remains a stated incumbent constraint: this paper does not exhibit which IPD reviews will host Explore closeout or Scale admission at Org-B, and it does not invent mid-September Explore closeout status.

Classical product Stage-Gate practice already separates phases with business-case reviews [10]. The incremental experience recorded here is narrower: an AI4SE admission contract that treats a stubbed executable chain as Consolidation work, publishes a withheld Scale decision beside a passed Explore closeout, and treats a second organization’s still-open Explore as recognition rather than as a completed case. We do not claim a new general Stage-Gate theory.

B. Manager Takeaways (Actionable)

- 1) **Next week (50+ engineer org).** Pick one delivery stream; freeze paradigm p and harness $\mathcal{H}^p$ (for Org-A, SDD-oriented); ban tool-only rollouts that skip a citable method and a readiness-labeled ledger.

- 2) **At the first executive readout.** Put Explore closeout and Scale admission on separate agenda lines. Ask the five Explore questions (citable method; countable opportunity assets; stub/ready labels; bounded signals; discussable Consolidation path) before any Scale calendar is debated.
- 3) **Playbook v1 clauses that flip first (Org-A lesson).** (i) guidance that “reads right” but has no runnable L3 path; (ii) promoting stub skills into autonomous agents; (iii) letting the apply path self-certify verify. Rewrite those before opening a second team.
- 4) **Before any Scale wave.** Name intent/spec/orchestration/verification owners, designate seed coaches, and ship a starter kit another team can run; keep unmet Scale-admission items on a public hold list.
- 5) **Keep negative evidence visible.** An unmet Scale admission, an in-progress Explore, and unaccepted deliverables are decision inputs—not narrative failures to hide.

C. Boundary with the Measurement Companion

This paper uses bounded effectiveness and maturity materials only as qualitative stage-criteria inputs (Org-A L4; Org-B Goal 5 as scheme design). The companion measurement paper’s unit of analysis is a dual-lens reporting scheme—labeled capability and delivery signals at Org-A—not the operating stack and adoption rhythm. The two manuscripts share a short anonymized Org-A context and the same enablement window; they do not share main figures, and we do not import that paper’s tables or treat its workshop or maturity numbers as results of the path.

VII. THREATS TO VALIDITY

External validity. The primary longitudinal case is a single organization (Org-A). Org-B is an in-progress Explore at manuscript freeze, not a balanced second site with a matching stage-criteria packet. Mode diversity inside Org-B improves within-organization coverage; it does not create a multi-enterprise sample.

Construct validity. “Scale-ready” is a criteria-and-asset construct in this paper, not a platform-measured return construct. Absence of completed rollout must not be read as proof that the method is ineffective, and bounded Explore signals must not be read as Scale completion.

Internal validity / observer bias. Authors facilitated enablement, stage-criteria framing, and asset authoring at Org-A, and they are guiding Org-B’s Explore. Consolidation and readiness judgments may reflect consultant–client alignment rather than independent audit. We mitigate by binding factual sentences to a frozen evidence pack, by publishing negative evidence (stubbed L3; unmet Scale admission; Org-B still in Explore), and by recording stage dispositions as client management decisions (Advance / Hold) rather than as consultant self-scores.

Org-B verification limits. Org-B prose relies on an SOW-aligned author briefing. The absence of an intranet artifact

mirror limits external checking of those claims. The adoption-after-alternatives account is given at sector and pattern level without naming providers, so readers cannot independently reconcile competitive context.

Conclusion validity. A positive Explore closeout plus a negative Scale admission is an internally consistent pair, not a contradictory evaluation. Treating either decision as a completed-scale result, or treating Org-B’s in-progress Explore as a second completed case, would be an invalid conclusion.

VIII. RELATED WORK AND CONCLUSION

A. Related Work

Methodological positioning. We follow software-engineering case-study guidance on context, chain of evidence, and threats [2], and we treat the operating stack and adoption rhythm as a designed enablement artifact [3] illustrated primarily at Org-A. We do not claim theoretical saturation or a completed design-science evaluate cycle with field-measured Scale outcomes.

Productivity caution, not a measurement contribution. Forsgren et al. argue that developer productivity is multi-dimensional and that single metrics mislead [1]. We take that caution as a reason not to treat a pilot slogan as Scale authorization. The industrial dual-lens reporting protocol that labels and jointly publishes capability and delivery signals is the companion paper’s contribution, not ours.

Sensemaking and classical SE layering as path rationale. Cynefin decision domains motivate treating early AI4SE work as Complex probes rather than Clear rollouts [4]. Classical layered software engineering [7] motivates keeping process/methods/tools structure while upgrading executors and placing harness/SDD/agentive assets primarily in Methods. Fowler’s harness writing supplies the bounded-context intuition we adapt for a user-harness certainty boundary [9]. None of these sources is re-evaluated here; they justify staging and placement vocabulary only.

Industry retreat synthesis (rationale taxonomy only). Thoughtworks’ Utah and Europe Future of Software Engineering retreats map practitioner fault lines—rigor migration into specs/tests/constraints, a supervisory middle loop, verification scarcity, harness ownership, and decision-time bottlenecks [5], [6]. We borrow only that *fault-line taxonomy* as staging rationale (why Explore evidence, Consolidation hardening, and conditional Scale admission are distinct decisions). We do not import retreat anecdotes as Org-A/Org-B facts, do not treat retreat consensus as multi-site validation of our stage criteria, and do not claim the retreats prescribe our five-question packet; SPACE remains our productivity caution [1].

Spec-driven and agentive practice. Gupta’s preprint discusses spec-driven development together with agentive frameworks [8]. Our use of SDD and Agentive Engineering is narrower: they are two directions inside a managed path, with harness and operating-model obligations attached to later stages.

Maturity models for AI-assisted SE. Tereci et al. propose a conceptual maturity model and research agenda for AI-

assisted software development [11]. Anderson’s experience-report preprint frames progression from assisted coding toward more autonomous systems [12]. Those works target organizational or codebase maturity theory. Our unit of analysis is a AI4SE operating stack with an adoption rhythm and the closeout and admission criteria of its stages, not a new maturity model.

Differentiation from generic Stage-Gate vocabulary. Classical Stage-Gate product processes separate discovery, development, and launch with business-case gates [10]. Our rhythm reuses the *idea* of evidence-based admission without adopting Stage-Gate vocabulary for the organizational layer—and the review objects differ: AI4SE admission inspects a citable method, a stub-versus-ready executable ledger, and transfer preconditions—not a product business case. Generic Stage-Gate vocabulary does not, by itself, prevent equating a coding-assistant pilot with organization-wide diffusion, nor does it encode stub $\neq$ ready for agent skill packs. Relative to AI-assisted maturity models, and to that generic practice, this paper records an AI4SE-specific pairing: a withheld Scale admission beside a completed Explore at Org-A, plus a thinner Org-B recognition vignette rather than a second completed asset ledger. We do not claim that pairing is unique in the literature; it is the experience we can evidence.

B. Conclusion

We asked how an industrial AI4SE path from pilot toward scale can be designed (Q1), what Org-A’s Explore produced in support of Consolidation and of only-conditional Scale (Q2), and what a still-open Org-B Explore can legitimately show about adoption after infeasible alternatives (Q3).

The transferable artifact is the operating stack and adoption rhythm: Explore, Consolidation, and conditional Scale, operated as evidence reviews at Explore closeout and Scale admission. Org-A shows that a six-week Explore can yield a citable method, countable opportunity knowledge, and a stub-versus-ready ledger, and that those results can justify Consolidation without authorizing Scale. Org-B contributes only a decision fact and an in-progress multi-mode Explore under IPD; it does not contribute a second completed stage-criteria packet. Neither organization is claimed as organization-wide scale; neither reports platform-measured return.

Future camera-ready updates may refresh Org-B’s mid-September delivery status once author-confirmed. They should not convert an unmet Scale admission, or an in-progress Explore, into a completed-scale claim.

ACKNOWLEDGMENT

We thank Org-A and Org-B engineering participants for stage reviews and enablement collaboration. Individuals and legal client entities remain unnamed under partner confidentiality; publication is anonymized without client co-authorship. Funding disclosures, if any, will be finalized for the camera-ready version.

DATA AVAILABILITY

Anonymized, self-contained evidence for this manuscript is released at <https://github.com/ai4se-works/artifacts/tree/main/manuscripts/stage-gated-ai4se-path/data> (CC-BY-4.0). The pack includes a stage-criteria rubric (Explore closeout / Scale admission), a public L1 method card (chapter map, principles, artifact chain, process Align/Verify/Merge reviews, paraphrased normative clauses, and declared v1 gaps), an L1–L4 asset ledger, a twelve-theme index with observation flags, a redacted L3 stub map, and the Org-A timeline used in the case narrative. It is intended for inspection of those claims without access to partner intranets. The full client-facing playbook text, opportunity Spec/BP bodies, plugin sources, unredacted change records, and Org-B workspace content remain confidential. Quantitative maturity and workshop tables are not reproduced here; they appear in the companion measurement data pack in the same artifacts repository.

REFERENCES

- [1] N. Forsgren, M.-A. Storey, C. Maddila, T. Zimmermann, B. Houck, and J. Butler, “The SPACE of developer productivity,” *Communications of the ACM*, vol. 64, no. 6, pp. 46–53, 2021.
- [2] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [3] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–106, 2004.
- [4] D. J. Snowden and M. E. Boone, “A leader’s framework for decision making,” *Harvard Business Review*, vol. 85, no. 11, pp. 68–76, 2007.
- [5] Thoughtworks, “The future of software engineering: Retreat findings and strategic insights,” Thoughtworks, Tech. Rep., Feb. 2026. [Online]. Available: https://www.thoughtworks.com/content/dam/thoughtworks/documents/report/tw_future%20of_software_development_retreat%20key_takeaways.pdf
- [6] —, “The future of software engineering: Insights and findings from an unconference on AI, agentic engineering and the future of the discipline,” Thoughtworks, Tech. Rep., Jun. 2026. [Online]. Available: https://www.thoughtworks.com/content/dam/thoughtworks/documents/report/tw_future_of_software_engineering_europe_2026.pdf
- [7] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 8th ed. McGraw-Hill Education, 2014.
- [8] N. Gupta, “Small language models and spec-driven development for high-accuracy agentic frameworks,” SSRN preprint, 2026.
- [9] M. Fowler, “Harness engineering for coding agent users,” Online essay, Apr. 2026. [Online]. Available: <https://martinfowler.com/articles/harness-engineering.html>
- [10] R. G. Cooper, “Stage-gate systems: A new tool for managing new products,” *Business Horizons*, vol. 33, no. 3, pp. 44–54, 1990.
- [11] S. Tereci, E. Gökalp, and A. Dikici, “Toward a maturity model for AI-assisted software development: Conceptual framework and research agenda,” in *2026 5th International Informatics and Software Engineering Conference (IISEC)*, 2026, pp. 656–661.
- [12] A. Anderson, “The AI codebase maturity model: From assisted coding to fully autonomous systems,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.09388>